USCO BEINSOF52 · Artificial Intelligence · Project C1

Hydroponic Lettuce Growth-Stage Classifier

Five deep-learning backbones · ensemble · FastAPI · React

Jesús Beleño · Juan Forero

May 2026

Is this pod ready to harvest?

Hydroponic operators face a recurrent perception task: every pod in a tray is at a different point of the growth timeline, and the right moment to harvest, re-sow or fertilize is decided pod-by-pod.

Doing this visually for 1,500+ pods a day is feasible but fatiguing. The cost of error compounds in both directions — harvest early and yield drops; harvest late and the plant bolts.

We trained an image classifier that returns the growth stage of one pod in under a second and persists every decision so the same farm can audit and retrain.

19,721

labeled pod crops

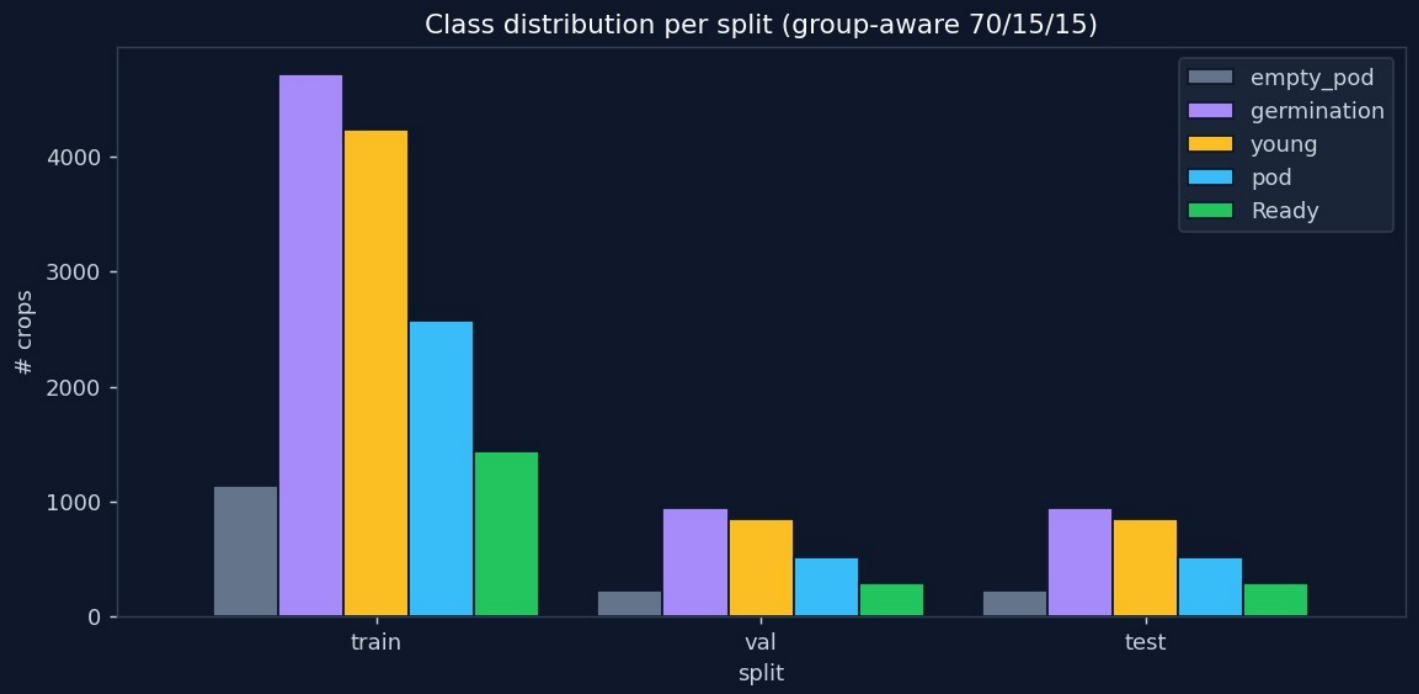
5

growth stages classified

93.86%

test accuracy (Swin-T)

Roboflow lettuce-pallets · 19,721 crops



SOURCE FRAMES

1,501

CROPS TOTAL

19,721

TRAIN · VAL · TEST

71.4 % · 14.3 % · 14.3 %

SPLIT STRATEGY

Stratified group-aware (no source-frame leak)

CLASS BALANCE

germ 33.5 % → empty 8.1 % (4.1×)

BALANCING IN LOSS

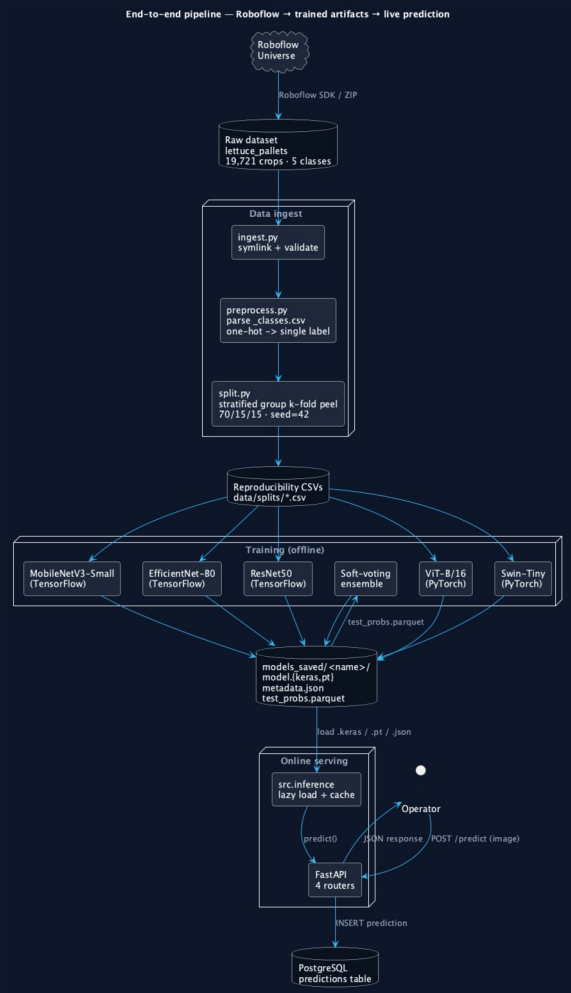
class_weight = "balanced"

Five ImageNet-pretrained networks compared

MODEL	FRAMEWORK	PARAMETERS	ROLE
MobileNetV3-Small	TensorFlow	~2.5 M	edge baseline
EfficientNet-B0	TensorFlow	~5.3 M	midsize CNN
ResNet50	TensorFlow	~25.6 M	canonical CNN
ViT-B/16	PyTorch	~86 M	global self-attention
Swin-Tiny	PyTorch	~28 M	hierarchical attention

Same head topology across the five: backbone → pooling → dropout (TF) → Dense(5, softmax) with dtype=float32 on the head so mixed precision stays numerically stable.

End-to-end pipeline



Two-phase fit for CNNs · end-to-end for transformers

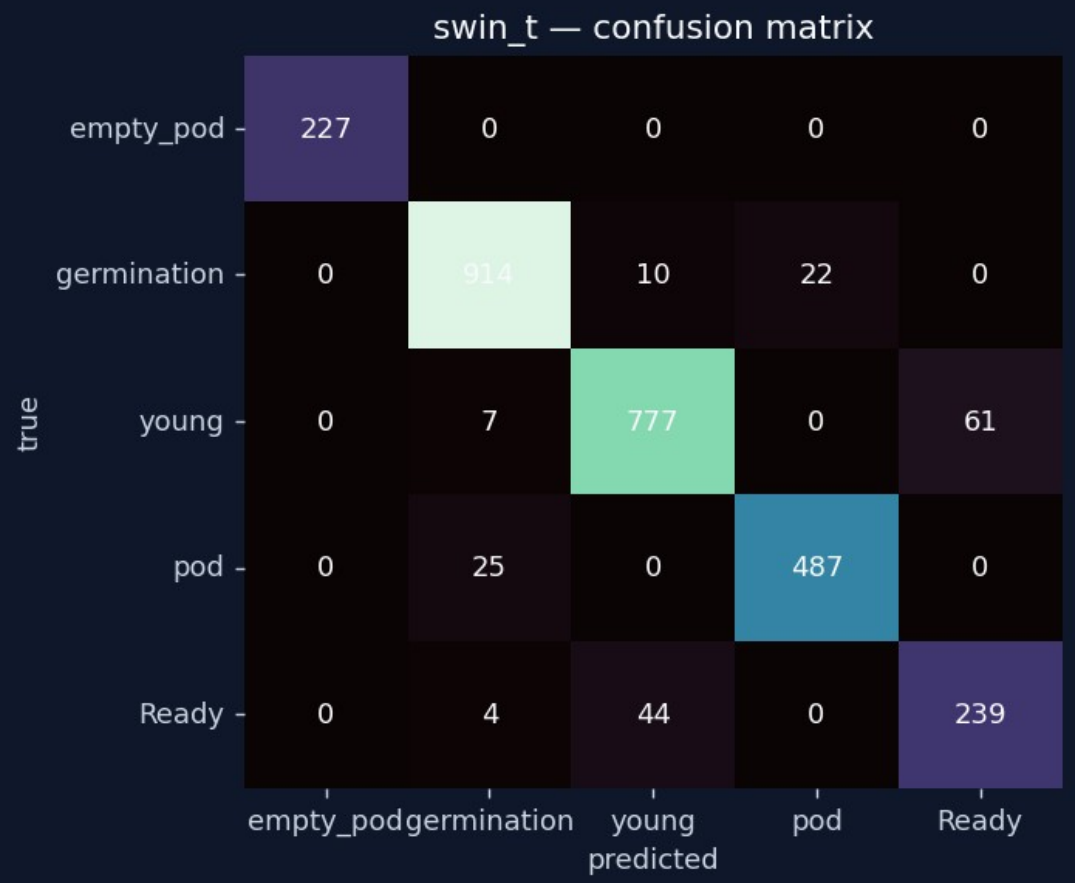
TF CNNs	**phase 1 — head only** 12 ep · Adam(1e-3)
TF CNNs	**phase 2 — top 30 % of backbone** 4 ep · Adam(1e-5)
TF CNNs	**BatchNorm always frozen** preserves running stats
Torch	**ViT-B/16 + Swin-T end-to-end** 8 ep · AdamW(3e-5)
All	**two-layer class balancing** sample-level oversampling (balance.py) + loss-level class_weight
All	**augmentation pipeline** flip · rotation · brightness · contrast · zoom
All	**mixed precision on CUDA** fp16 forward · fp32 softmax/loss
All	**checkpoints on best val_acc** ModelCheckpoint + EarlyStopping (patience 3)

Held-out test set comparison

MODEL	ACC.	MACRO F1	MACRO RECALL	BEST VAL
swin_t	93.86%	93.19%	93.39%	94.03%
vit_b_16	93.65%	92.81%	92.57%	94.28%
ensemble_avg	93.65%	93.22%	94.06%	—
resnet50	90.98%	90.56%	91.72%	90.41%
efficientnet_b0	89.39%	89.32%	91.78%	88.88%
mobilenet_v3_small	88.32%	88.55%	90.19%	88.28%

Swin-Tiny wins overall accuracy. The ensemble wins macro-recall — the metric aligned with operational cost (every missed Ready costs harvest timing; every missed empty_pod costs re-sowing).

Where does Swin-Tiny still confuse?



`young → pod`

the most expensive confusion. Visually similar mid-stage canopies.

`empty_pod`

highest per-class precision. Visually distinct (no plant).

`Ready`

high recall. The operationally critical class is well caught.

`germination`

isolated. Smallest class but most distinct early-stage cue.

FastAPI · PostgreSQL · React · Vite

ENDPOINTS

POST /predict	upload → predict → persist
GET /history	paginated, label/model filters
GET /metrics	model-comparison table
GET /models	what is loadable right now

ENGINEERING

Lazy model loading

TF + Torch share one process · RLock so the ensemble loader doesn't self-deadlock

TF pinned to CPU

PyTorch keeps the GPU; no Metal/CUDA contention

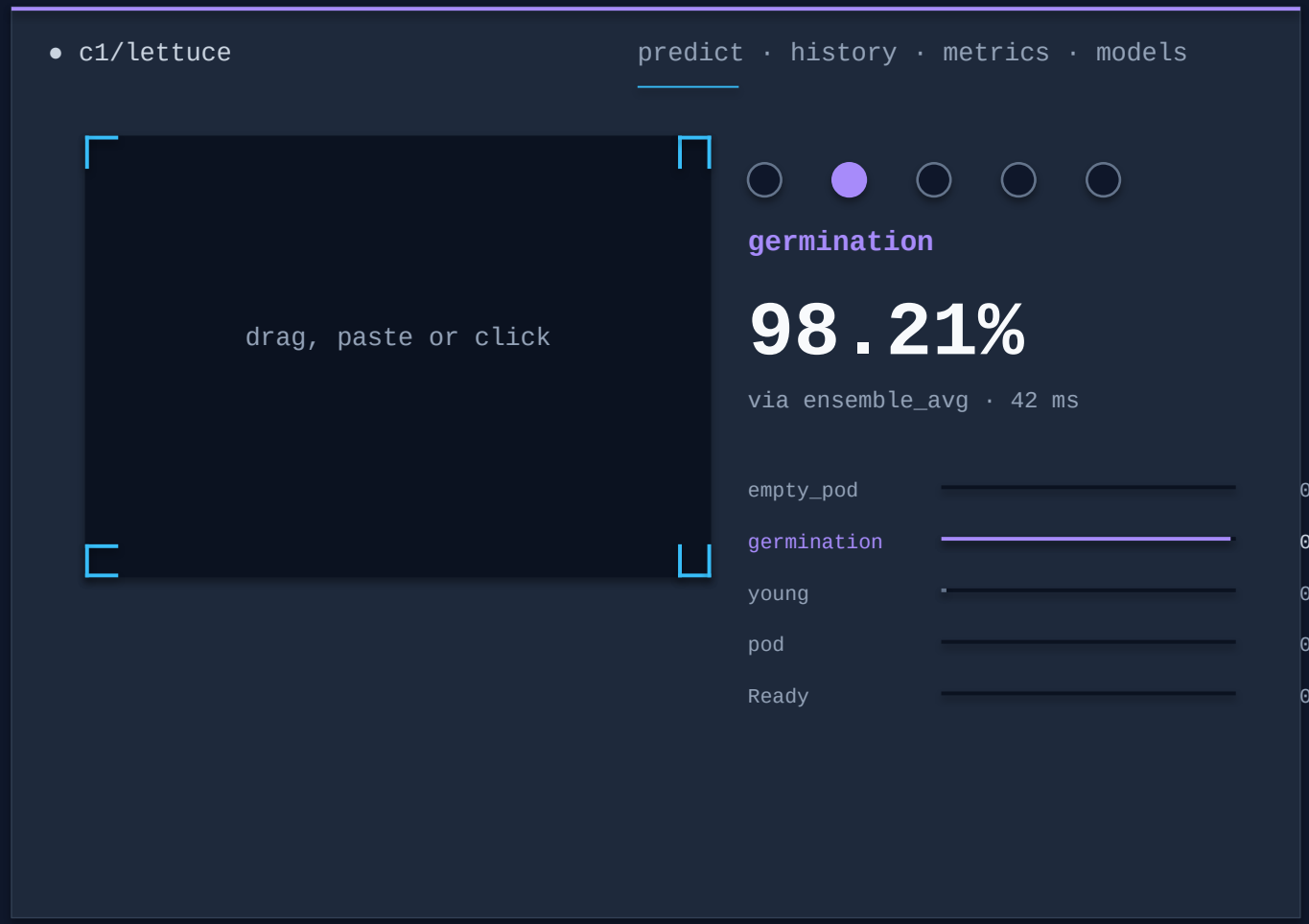
StaticPool SQLite

in-memory database in tests · production runs on PostgreSQL

Vite proxy

/api/* → 127.0.0.1:8000 — no CORS in dev

React dashboard · webcam · viewfinder crop



- Vanilla getUserMedia for the laptop webcam — no camera library
- Drag-and-drop, clipboard paste or click on the viewfinder
- Crop step between picking and predicting — one pod per request
- The GrowthSpine signature runs across all four routes
- Single accent per card = the predicted class color, never two

A loss of 1,191 on the very first MobileNet run

The smell test

MobileNetV3 was passed include_preprocessing=False with no manual normalisation, while modern Keras preprocess_input for MobileNetV3 is a no-op. The network was fed pixels in [0, 255] when its ImageNet weights expected [-1, 1].

Loss exploded into the thousands; test accuracy capped at 0.35 — barely above the 0.20 random baseline. The fix was one flag.

BEFORE

test_acc 0.3465
loss epoch 1: 1190.88

AFTER

test_acc 0.8832
loss epoch 1: 0.59

One line of code · 4 min training
+0.53 accuracy

Engineering and product directions

Detection layer

Add YOLOv8 in front of the classifier. The Roboflow source is annotated with boxes already — fine-tune on the same data, then run our 5-model classifier on each detected box. Closes the multi-pod gap end-to-end.

Weighted ensemble + stacking

Uniform soft-voting underweights the transformers. Weighted soft voting (val-accuracy proportional) or a logistic-regression meta-learner trained on out-of-fold softmax vectors.

K-fold CV in the report

make cv-swin already exists and writes reports/cv_swin_t/. Attach a mean ± std to the headline 0.9386 to silence the "lucky split" question.

Cloud deployment

Dockerize the FastAPI service + Postgres, push to Render free tier, get a public https URL the operator opens on a tablet.

QUESTIONS · DISCUSSION

Thanks.

github.com/jbeleno/c1-lettuce-classifier

Jesús Beleño · Juan Forero